

IronClaw: Dynamic Tool Synthesis and Secure MicroVM Isolation for Autonomous LLM Agents

Suryavir Kapur and Vijay Kumar Balakrishnan
Department of Computer Science & Information Systems
Birla Institute of Technology and Science Pilani
Email: f20220293@dubai.bits-pilani.ac.in

Abstract—Large Language Model (LLM) agents increasingly execute complex multi-step workflows. When no purpose-built capability exists, an agent can either author the required logic inline on every request or synthesize a task-specific tool once and reuse it. Reuse promises efficiency gains, but executing synthesized code in an ordinary process creates significant security risk. This paper presents IronClaw, an autonomous-agent platform that combines dynamic tool synthesis with strictly isolated Firecracker microVMs. A trusted host daemon manages agent policy, context, and lifecycle, while an ephemeral guest runtime executes generated code behind a hardware-virtualization boundary, reachable only over an authenticated protobuf/vsock channel. We evaluate the two resulting claims independently. On the utility side, across SQL, log-parser, and repository-compliance domains, registering and reusing synthesized tools reduced billed input tokens by 53.4%, 61.4%, and 53.6% on repetitive workloads relative to inline authoring, with 100% answer correctness in both conditions; on mixed workloads with only three-of-six reuse, synthesis was net-negative (+19.0% and +64.7% input tokens). On the security side, all five host-resource attacks succeeded under ordinary process execution and none succeeded inside Firecracker; containment cost a 6.08s median cold start, a 6.62s median recovery, and 72,672 KiB host resident memory for a 512 MiB guest, while warm dispatch over an existing session cost only 1.74ms. Dynamic synthesis therefore pays off exactly when reuse is expected, and microVM isolation is practical today for long-lived agent sessions; per-request isolation awaits snapshot restore.

Index Terms—autonomous agents, large language models, dynamic tool synthesis, Firecracker, microVM, sandboxing, isolation, vsock

I. INTRODUCTION

LLM-based autonomous agents are being applied to long-horizon tasks in software engineering, data management, document processing, and process automation. Most deployed agents operate with a *static registry* of predefined tools chosen by their developer before the task begins. Static registries simplify auditing and permissioning, but they cannot cover the intermediate parsers, converters, checks, and integrations that open-ended tasks routinely demand.

Dynamic tool synthesis (DTS) removes this limitation: the agent writes a small program, executes it, observes the result, and registers it as a reusable capability. This changes the security model. Generated code can contain destructive operations, command-injection payloads, resource-exhaustion loops,

or deliberate host escapes, whether by model error or prompt injection. Sandboxing only the process that hosts the agent is therefore insufficient: agent-generated code must be treated as adversarial regardless of how benign the user’s request appears.

IronClaw addresses this joint utility/security problem by pairing DTS with Firecracker microVM isolation. A trusted host daemon owns the agent loop, policy, memory, LLM access, and VM lifecycle; an intentionally small, ephemeral guest runtime executes untrusted tool code inside a hardware-virtualization boundary and returns only bounded, structured results over an authenticated vsock channel. The design lets us measure both sides of the trade-off with matched baselines, guided by two research questions:

- **RQ1 (Utility).** On repeated tasks with no purpose-built tool, does dynamic tool synthesis reduce model interaction steps and token consumption relative to authoring the logic inline, without reducing correctness?
- **RQ2 (Security/Systems).** What containment capability and performance overhead does Firecracker microVM isolation provide compared with process-level execution of agent-generated code?

Our contributions are:

- 1) **An architecture** that integrates dynamic tool synthesis with ephemeral microVM execution behind a strict host/guest boundary, implemented as a Rust workspace (host daemon, guest runtime, shared protocol/transport crates) with process, container, and microVM sandbox backends.
- 2) **A utility evaluation** across three domains (SQL synthesis, legacy-log parsing, repository compliance) showing that reuse of synthesized tools cuts billed input tokens by 53.4–61.4% and model steps by up to 46.4% on repetitive workloads while preserving 100% correctness—and that the same mechanism is net-negative when reuse is sparse.
- 3) **A security and performance evaluation** showing 0/5 successful host-resource attacks inside Firecracker versus 5/5 under process execution, with measured costs: 6.08s median cold start, 6.62s median recovery, 72,672 KiB host resident memory, and 1.74ms warm dispatch.

Section II reviews background and related work. Section III

presents the IronClaw architecture. Section IV describes the evaluation methodology. Sections V and VI report the utility and security/performance results. Section VII discusses limitations, and Section VIII concludes.

II. BACKGROUND AND RELATED WORK

A. LLM Agents and Tool Use

LLM agents extend language models with external tool calls, task state, and multi-step planning, delegating exact computation and program execution to deterministic systems. ReAct showed that interleaving reasoning traces with actions improves interactive task solving [1]; Toolformer showed that models can learn when external API calls are useful [2]; Generative Agents demonstrated memory, reflection, and planning loops [3]; and Voyager showed an agent accumulating executable skills in an open-ended environment [4]. Despite this progress, most deployed agents still rely on static tool registries, which eases auditing but prevents runtime capability creation. Benchmarks such as SWE-bench evaluate reasoning over real repositories [5]; our evaluation instead targets the tasks most likely to benefit from tool generation—parsing, checking, transforming, and repeated verification.

B. Dynamic Tool Synthesis

DTS targets work whose required code cannot be predicted in advance: converting unknown file formats, aggregating benchmark outputs, validating configurations, or building domain-specific checks. The agent authors code, runs it in an isolated environment, inspects the output, and then revises or reuses the artifact. Our utility experiments quantify when this create-once, reuse-many pattern is worth its creation cost.

C. Isolation: Processes, Containers, VMs, MicroVMs

Process isolation relies on the host OS memory and permission model: all processes share the host kernel, so a kernel bug or misconfiguration can escalate a process compromise into a host compromise. Containers add namespaces, cgroups, and capabilities but still share the host kernel. Virtual machines run a guest OS on virtualized hardware, so privileged operations are mediated by a hypervisor, satisfying the classic VMM goals of equivalence, resource control, and efficiency [6]; Xen is a canonical example [7]. Hardware extensions (Intel VT-x/AMD-V) run guest code directly on the CPU, trapping sensitive operations via VM exits to the hypervisor [8], [9], while Extended Page Tables enforce memory isolation between guests and host. MicroVMs keep the hardware boundary but shrink the device model and management surface, making them suitable for short-lived isolated workloads; Firecracker is a production implementation designed for lightweight serverless virtualization [10]. Figure 1 summarizes this continuum. IronClaw creates a microVM per task session, runs the generated code, returns bounded output, and discards the guest, preventing persistence and cross-tool contamination.

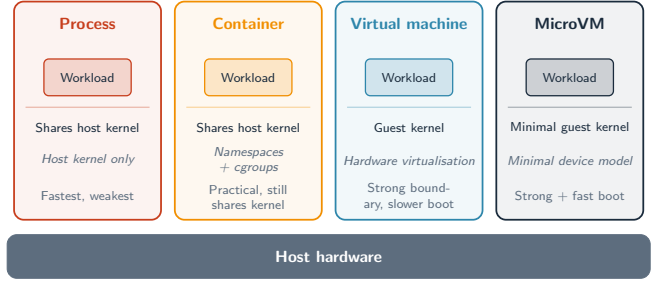


Fig. 1. Isolation continuum: process, container, VM, and microVM boundaries. Only VM-based boundaries remove the shared-kernel dependency.

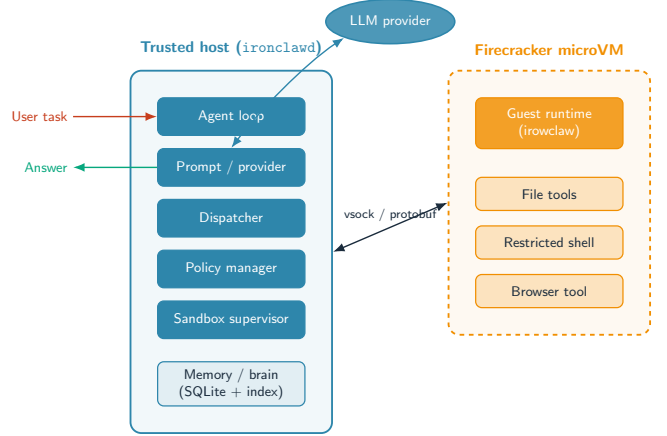


Fig. 2. IronClaw host/guest architecture. The trusted host daemon runs the agent loop and owns the sandbox lifecycle; generated tools execute inside a Firecracker microVM guest runtime.

III. IRONCLAW DESIGN

A. Asymmetric Host/Guest Split

IronClaw is built around a strict split of trust (Figure 2). The host daemon (*ironclawd*) is trusted and stateful: it loads configuration, enforces policy, manages memory, talks to LLM providers, and controls the microVM lifecycle. The guest runtime (*irowclaw*) is untrusted and ephemeral: it accepts bounded execution requests, runs them under local limits, returns structured results, and can be destroyed at the end of a session. Because generated tools are untrusted, the host never executes them; it packages the tool specification and code for the guest.

Five host subsystems cooperate: (i) an *agent loop* maintaining the plan–act–observe cycle and deciding whether the next action is a fixed tool call, a generated tool, or the final answer; (ii) a *prompt/provider layer* that builds prompts from task context, memory, tool results, and policy reminders; (iii) a *dispatcher* that normalizes model actions into typed execution requests and routes them to local fixed tools or the guest; (iv) a *policy manager* that bounds filesystem access, network access, CPU time, memory, output size, and allowed interpreters; and (v) a *sandbox supervisor* that creates, monitors, times out, and destroys guest runtimes. The host also performs *result*

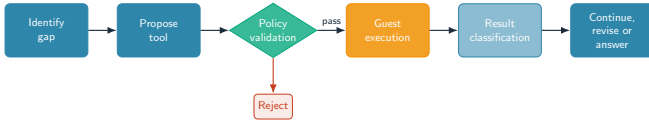


Fig. 3. Dynamic tool synthesis flow, showing the policy gate between tool proposal and guest execution.

triage: guest responses are bounded, decoded, tagged with status metadata, and summarized where necessary, so large or malformed outputs cannot destabilize the next agent turn.

The guest runtime is deliberately minimal. It validates the request envelope (request identifier, tool name, input payload, execution limits), materializes the execution environment (input files, interpreter selection, local process limits), and captures a structured response (exit status, stdout, stderr, duration, error classification). It holds no secrets, credentials, or host mounts: every byte needed for an execution is explicitly sent by the host, so a payload inspecting the system sees only the minimal guest filesystem and the inputs for that single run.

B. Transport Boundary and Protocol

Host and guest communicate over a controlled channel—an authenticated protobuf envelope carried over Firecracker vsock (with in-memory and child-process transports reused for testing). Each envelope carries `user_id`, `session_id`, `msg_id`, `timestamp_ms`, and a `cap_token`; the payload is a `oneof` covering authentication (`AuthChallenge/AuthAck`), agent control, user messages, tool calls/responses, file operations, and scheduler events. Before execution, the host performs a capability handshake: it sends a capability token and a tool allow-list, and the guest filters its registry to only those tools. A request envelope is explicit rather than shell-like: request id, tool name and type, code or command material, serialized inputs, policy fields (timeout, memory, network mode, output limit), and the expected result format. Responses mirror this structure with status, exit code, duration, bounded stdout/stderr, and artifact metadata, letting the host distinguish timeout, policy denial, runtime crash, and success instead of treating every reply as plain text.

C. Dynamic Execution Engine

Tool generation and tool execution are kept separate: the model may propose code, but the host decides eligibility and the guest confines the consequences. The loop proceeds in six stages (Figure 3): (1) the agent identifies that no fixed tool suffices; (2) the model proposes a small tool with purpose, inputs, and code; (3) the host validates the specification against policy and rejects unsafe requests before guest execution; (4) the guest runs the tool in the microVM and captures bounded outputs; (5) the host classifies the result as success, tool error, policy error, timeout, or infrastructure error; (6) the agent continues, revises the tool, or answers.

D. Ephemeral Sandboxing and Failure Model

Each session runs in an ephemeral microVM: allocate/resume the guest image, establish vsock, send the request bundle, enforce host-side timeout and output limits, receive and validate the response, copy out approved artifacts, then destroy or reset guest state. This blocks cross-tool contamination, persistent state abuse, and memory poisoning. Snapshot resume can reduce startup time but must represent a clean base state, untainted by task-specific secrets or previously generated tools. Every failure is classified (tool, policy, timeout, transport), so the next agent step proceeds without exposing the host to the failing execution.

E. Implementation

The system is a Rust workspace¹ separating the control plane from the execution plane: `ironclawd` (host daemon, agent loop, sandbox managers), `ironclaw` (guest runtime), `common` (configuration, protobuf bindings, codec, transport traits, Firecracker abstractions), `tools` (workspace-rooted file tools, restricted shell, headless browser), `memory` (SQLite store with hybrid retrieval and redaction), `security` (authentication, pairing, rate limiting), and `channels`. The host supports process, container, and microvm sandbox backends behind one manager abstraction, which lets the same agent loop be evaluated under different containment strategies. Firecracker guests boot a minimal kernel command line (`panic=1,pci=off,nomodules,root=/dev/vda,init=/init`); after `InstanceStart`, the host connects over the vsock UDS, issues `CONNECT`, and wraps the stream in the same framed transport used elsewhere. Guest-side safety checks (path rooting, shell sub-string denylists, output caps, scheme validation) provide defense in depth *within* the guest—they complement, and do not replace, the virtualization boundary.

IV. EVALUATION METHODOLOGY

The evaluation measures utility, security, and systems performance independently, with matched baselines for each hypothesis.

A. Utility (H1): No Synthesis vs. Dynamic Synthesis

Two scenarios share the same generic base capabilities and differ only in whether the repeated artifact can be registered for reuse. **Scenario A (no synthesis)**: the agent must author the required task logic inline for every request. **Scenario B (dynamic synthesis)**: the same capabilities plus a meta-tool and explicit policy to register the repeated logic once and reuse it. The comparison holds data, requests, model, and generic capabilities constant. The harness is a deterministic TypeScript experiment suite built on the Vercel AI SDK and its `dynamicTool()` primitive [11]; all conversations use `deepseek-v4-pro`, a reasoning-enabled model [12].

Five experiments cover three domains:

- **SQL (E1/E2)**. An airline customer-service agent answers cancellation questions over a seeded SQLite database of

¹Source: <https://github.com/suryavirkapur/ironclaw>

seven tables (150 passengers, 220 bookings); every answer requires a seven-table join of roughly one page of SQL, and the schema is not disclosed in advance.

- **Parser (E3/E4).** An operations analyst extracts cancellation events from six daily log exports in a hostile legacy format: arbitrarily ordered `key=value` fields, quoted values containing commas, mixed USD/EUR amounts with a per-file exchange rate, and noise lines. Scripts run in a `node:vm` sandbox with an allow-listed `readFile(name)` capability, a one-second timeout, and bounded output—mirroring IronClaw’s execution envelope.
- **Repository compliance (E5).** A supply-chain agent audits six repository snapshots against seven deterministic policies (dependency versions, production secrets, container users and health checks, mutable CI action references, Node versions, debug settings).

Repetitive workloads (E1, E3, E5) issue six requests that all reuse the same logic with different inputs; *mixed* workloads (E2, E4) reuse it in only three of six. Metrics per request: billed input/output tokens (summed over all model steps), reasoning tokens, model steps, tool calls, characters of authored code, wall-clock time, and correctness verified against ground truth.

B. Security and Performance (H2): Process vs. MicroVM

The same generated shell payloads run under two boundaries. **Process baseline:** a child shell on the host with the invoking user’s view of the filesystem, environment, devices, and process namespace. **Firecracker condition:** the identical command executed by IronClaw’s unrestricted guest tool inside a microVM, reached through the authenticated protobuf/vsock path; no host directory or environment variable is mounted into the guest. Five non-destructive payloads attempt: (1) reading a host-only sentinel file, (2) writing a marker into a host-only directory, (3) reading a host-only environment variable, (4) detecting the host’s `/dev/kvm`, and (5) reading the host hostname via `/proc`. An attack succeeds only if the targeted *host* resource is actually observed or modified; the acceptance criterion is 0/5 successful accesses in the microVM condition.

Performance is measured independently of model inference: 20 process-creation samples, 10 cold microVM starts (fresh copy-on-write tenant disks), 20 warm no-op executions over an existing authenticated vsock session, five stop–restart–authenticate–probe recovery trials, host resident memory from `/proc`, and CPU utilization around a bounded shell loop. The initialization target is <500 ms for *snapshot-resumed* guests; cold boot is reported separately. The host ran Linux 7.0.12 on an Intel Core i7-12700K; guests used Firecracker 1.14.0, two vCPUs, 512 MiB configured memory, a Linux 6.1.155 kernel, and an Ubuntu 24.04 root filesystem. Networking was disabled to keep the comparison controlled; network-policy containment is therefore out of scope for these experiments.

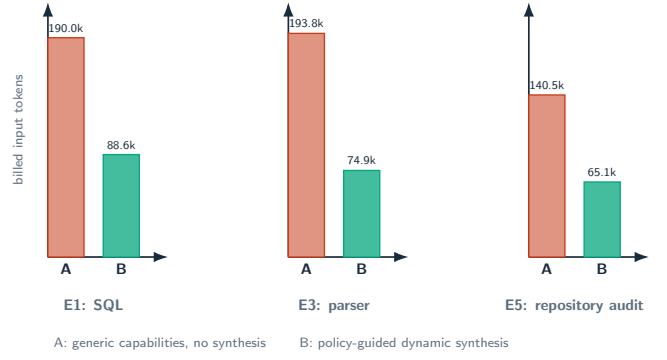


Fig. 4. Billed input tokens for no synthesis (A) and dynamic synthesis (B). Repetitive workloads (E1, E3, E5) save 53–61%; mixed workloads (E2, E4) are net-negative.

V. UTILITY EVALUATION

A. Repetitive Workloads: Synthesis Wins Decisively

Table I reports totals over the six requests of each repetitive experiment. Dynamic synthesis reduced billed input tokens by 53.4% (SQL), 61.4% (parser), and 53.6% (repository compliance), with identical 100% correctness in both scenarios. In the steady state (requests 2–6, after the one-time creation), the effect is larger: for SQL, average input tokens per request fell from 35,183 to 14,840 (–57.8%) and authored code from 510 characters to *zero*—scenario B answers each passenger with a single two-step tool call while scenario A re-authors the query every time. For repository compliance, steady-state input tokens fell from 27,276 to 12,270 (–55.0%), output tokens from 2,049 to 583 (–71.6%), authored code from 4,818 to 901 characters, and wall time from 21.6 s to 7.7 s; requests 3–6 required no further authored audit code.

B. Mixed Workloads: Creation Overhead Dominates

When only three of six requests reuse the capability, the one-time creation cost is not amortized (Table II): scenario B lands at +19.0% (SQL) and +64.7% (parser) billed input tokens. Creating a tool costs extra code and model steps; the creation transcript then inflates the context of every later request, and with only two reuses the saved inline work does not repay it. In these workloads 6/6 reuse wins and 3/6 loses; the exact break-even lies between them and is task- and model-dependent.

C. Why the Mechanism Works

Three findings explain the effect. (i) **Synthesis offloads reasoning, not just tokens:** reasoning tokens dropped 76.2% (parser) and 71.8% (repository) because deterministic code substitutes for repeated model reasoning. (ii) **The enabling condition is tool-side data access:** in an initial parser configuration where created tools received the full log text as an argument, every call pushed bulk data through the conversation and synthesis was net-negative (+12% input); letting the sandbox load data itself through an allow-listed `readFile(name)` capability made tool inputs tiny and flipped the result to –61.4%. Tools must take lookup keys, not bulk data. (iii) **Runtime**

TABLE I

REPETITIVE WORKLOADS (E1, E3, E5): TOTALS OVER SIX REQUESTS. A = NO SYNTHESIS, B = DYNAMIC SYNTHESIS; Δ RELATIVE TO A. CORRECTNESS WAS 100% IN ALL CELLS OF BOTH SCENARIOS.

Domain	Input tokens (billed)			Model steps		Code authored (chars)		Wall time (s)	
	A	B	Δ	A	B	A	B	A	B
E1: SQL query synthesis	190,031	88,566	-53.4%	28	15	3,142	1,800	103.6	81.3
E3: JS parser synthesis	193,817	74,865	-61.4%	18	13	6,109	1,196	91.7	47.2
E5: Repository compliance	140,472	65,146	-53.6%	19	13	29,176	9,112	134.0	61.4

TABLE II

MIXED WORKLOADS (E2, E4): BILLED INPUT TOKENS OVER SIX REQUESTS WITH THREE-OF-SIX REUSE.

Domain	A: no synthesis	B: dynamic	Δ
E2: SQL	73,452	87,442	+19.0%
E4: parser	64,452	106,131	+64.7%

TABLE III

HOST-RESOURCE ATTACK OUTCOMES (FIVE NON-DESTRUCTIVE PAYLOADS).

Payload	Process	MicroVM	Contained
Host file read	succeeded	failed	yes
Host file write	succeeded	failed	yes
Host environment read	succeeded	failed	yes
Host device access	succeeded	failed	yes
Host namespace observe	succeeded	failed	yes

registration is per-step: a tool created mid-generation only becomes callable at the next model step, so the harness must step the tool loop explicitly—platforms exposing meta-tools must do the same.

D. Answer to RQ1/H1

H1 is supported within the tested conditions: when the same complex operation recurs across all six requests, dynamic synthesis preserves correctness while substantially reducing input tokens (53.4–61.4%), model steps (28→15, 18→13, 19→13), authored code, and wall time. It is *not* universally beneficial: at three-of-six reuse, input-token cost rises by 19.0–64.7%. Both scenarios achieved 100% correctness, so the claim is efficiency at preserved correctness, not a higher success rate.

VI. SECURITY AND PERFORMANCE EVALUATION

A. Containment: 5/5 vs. 0/5

Table III shows the attack outcomes, and Figure 5 the boundary under test. The process baseline exposed the targeted host resource in all five cases; the Firecracker guest exposed it in none. The file and environment sentinels were absent inside the guest, the host marker was never created, `/dev/kvm` was not visible, and the guest hostname differed from the host’s. This supports the containment component of H2 for the evaluated attack classes.

B. Performance: Cold Start Dominates, Warm Path Is Cheap

Table IV summarizes the measurements. Cold initialization is the dominant overhead: a 6,083.67 ms median, roughly 167× the

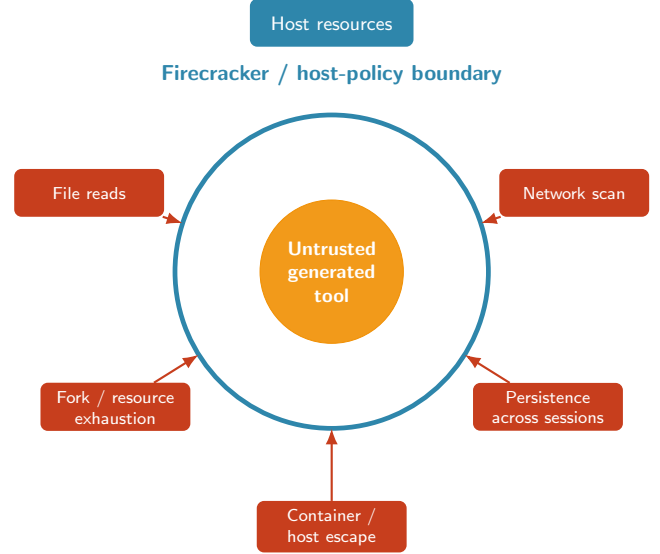


Fig. 5. Containment boundary for malicious synthesized tools: attack vectors are blocked by the Firecracker and host-policy boundary.

36.45 ms process-creation median. The Firecracker host process idled at 72,672 KiB resident memory—about 67 MiB above the process probe—with a 512 MiB configured guest-memory ceiling. Once a VM and vsock session exist, tool dispatch is inexpensive: the 1.74 ms warm median shows the steady state is not virtualization-bound. CPU utilization during the bounded loop was comparable (92.2% vs. 99.6%); absolute loop times are not compared because host and guest userlands differ. The recovery path—deliberate disposal, fresh boot, authentication, and a successful probe—succeeded in all five trials at a 6.62 s median: acceptable for replacing a failed long-lived tenant session, but not for per-request isolation with interactive latency requirements.

C. Answer to RQ2/H2

RQ2 has two answers. First, Firecracker provided complete containment for the defined suite: 5/5 payloads reached their target under process execution, 0/5 inside the microVM. Second, the boundary carries substantial cold-start and memory cost but a negligible warm-path cost. H2 is therefore *partially supported*: containment is demonstrated and the overhead is operationally bounded for long-lived VMs, but the “acceptable initialization” claim does not hold for per-request creation. The <500 ms

TABLE IV
PROCESS VS. FIRECRACKER PERFORMANCE (MEDIAN, P95 WHERE NOTED).

Metric	Process	Firecracker
Start/cold start	36.45 ms	6,083.67 ms
p95	94.00 ms	6,102.59 ms
Warm no-op median	36.45 ms	1.74 ms
p95	94.00 ms	3.68 ms
Host resident memory	4,108 KiB	72,672 KiB
Configured guest memory	—	512 MiB
CPU-loop utilization	92.2%	99.6%
Stop+restart+probe	—	6,622.87 ms
p95	—	6,632.47 ms

target applies to snapshot resume, which IronClaw does not yet implement; it remains unevaluated rather than failed.

VII. DISCUSSION AND LIMITATIONS

The two evaluations compose into a clear operating rule for IronClaw: *synthesize when repeated reuse is expected; otherwise execute inline; and keep bulk data behind the tool boundary*. Long-lived tenant sessions already satisfy both sides of the trade-off: one cold start amortizes over many tool calls, each warm dispatch costs milliseconds, and every synthesized artifact executes behind a hardware boundary that contained every attack we mounted.

The evidence has bounded scope. The utility study uses a single model (deepseek-v4-pro), three synthetic domains, scripted requests, and one measured run per condition; absolute numbers will vary with model, prompt, and data, and inline-authoring agents exhibit substantial strategy variance across runs. It identifies the direction and rough boundary of the reuse effect, not a universal break-even point between three and six repetitions. The security study uses one host, one Firecracker/kernel/rootfs configuration, and five deliberately non-destructive payloads against an *unisolated process* baseline; it does not cover network egress, kernel or VMM fuzzing, side channels, disk exhaustion, or concurrent adversarial workloads, and it does not establish superiority over stronger software sandboxes such as namespaces, seccomp, containers, or WebAssembly. Containment of the defined resource classes is demonstrated; absence of vulnerabilities is not claimed.

VIII. CONCLUSION AND FUTURE WORK

We presented IronClaw, an agent platform that combines dynamic tool synthesis with Firecracker microVM isolation behind a strict host/guest boundary. Our experiments show the two halves of the design are each measurable: reuse of synthesized tools reduces billed input tokens by 53.4–61.4% and model steps by up to 46.4% on repetitive workloads at unchanged correctness, while imposing a net cost when reuse is sparse; and the microVM boundary contained all five host-resource attacks that succeed under ordinary process execution, at a 6.08 s cold start and 72,672 KiB resident memory, with a 1.74 ms warm dispatch path. Four directions follow: (i) implement and measure snapshot create/restore against the <500 ms resume target, enabling per-request isolation; (ii) complete TAP-layer network

allow-lists and stronger CPU/memory/time/disk/output enforcement, and red-team them; (iii) broaden the utility benchmarks to real codebases, more models, repeated trials, more reuse frequencies, and cross-session tool caching; and (iv) extend the security suite with fuzzing, kernel/VMM instrumentation, side-channel analysis, and container/seccomp/WebAssembly baselines.

REFERENCES

- [1] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, “React: Synergizing reasoning and acting in language models,” in *International Conference on Learning Representations*, 2023, https://openreview.net/forum?id=WE_vluYUL-X.
- [2] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, “Toolformer: Language models can teach themselves to use tools,” in *Advances in Neural Information Processing Systems*, 2023, <https://arxiv.org/abs/2302.04761>.
- [3] J. S. Park, J. C. O’Brien, C. J. Cai, M. R. Morris, P. Liang, and M. S. Bernstein, “Generative agents: Interactive simulacra of human behavior,” in *Proceedings of the 36th Annual ACM Symposium on User Interface Software and Technology*, 2023, <https://dl.acm.org/doi/10.1145/3586183.3606763>.
- [4] G. Wang, Y. Xie, Y. Jiang, A. Mandlkar, C. Xiao, Y. Zhu, L. Fan, and A. Anandkumar, “Voyager: An open-ended embodied agent with large language models,” *Transactions on Machine Learning Research*, 2024, <https://openreview.net/forum?id=ehfRiF0R3a>.
- [5] C. E. Jimenez, J. Yang, A. Wettig, S. Yao, K. Pei, O. Press, and K. Narasimhan, “Swe-bench: Can language models resolve real-world github issues?” in *International Conference on Learning Representations*, 2024, <https://arxiv.org/abs/2310.06770>.
- [6] G. J. Popek and R. P. Goldberg, “Formal requirements for virtualizable third generation architectures,” *Communications of the ACM*, vol. 17, no. 7, pp. 412–421, 1974, <https://dl.acm.org/doi/10.1145/361011.361073>.
- [7] P. Barham, B. Dragovic, K. Fraser, S. Hand, T. Harris, A. Ho, R. Neugebauer, I. Pratt, and A. Warfield, “Xen and the art of virtualization,” in *Proceedings of the 19th ACM Symposium on Operating Systems Principles*, 2003, pp. 164–177, <https://dl.acm.org/doi/10.1145/945445.945462>.
- [8] K. Adams and O. Agesen, “A comparison of software and hardware techniques for x86 virtualization,” in *Proceedings of the 12th International Conference on Architectural Support for Programming Languages and Operating Systems*, 2006, pp. 2–13, <https://dl.acm.org/doi/10.1145/1168917.1168860>.
- [9] Intel Corporation, “Intel 64 and ia-32 architectures software developer’s manual,” 2025, <https://www.intel.com/content/www/us/en/developer/articles/technical/intel-sdm.html>.
- [10] A. Agache, M. Brooker, A. Iordache, A. Liguori, R. Neugebauer, P. Piwonka, and D.-M. Popa, “Firecracker: Lightweight virtualization for serverless applications,” in *17th USENIX Symposium on Networked Systems Design and Implementation*, 2020, pp. 419–434, <https://www.usenix.org/conference/nsdi20/presentation/agache>.
- [11] Vercel, “Ai sdk by vercel: Tool calling and `dynamicTool()`,” 2026, <https://ai-sdk.dev/docs/ai-sdk-core/tools-and-tool-calling>.
- [12] DeepSeek, “Deepseek api documentation,” 2026, <https://api-docs.deepseek.com/>.